MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION

GOVERNMENT POLYTECHNIC RATNAGIRI

2024-2025

A PROJECT REPORT ON –

JARVIS : THE AI ASSISTANT

SUBMITTED TO :

"COMPUTER ENGINEERING DEPARTMENT"

UNDER THE GUIDANCE OF:

"G.D. PATNE"

SUBMITTED BY:

- 3213- SAHEEM SAMEER NAKHWA (2200130075)
- 3229- SHWET ROHIT KADAM (2200130095)
- 3231- VEDANT VAIBHAV SURVE (2200130097)
- 3254- VIRAJ SATISH SHINDE (2200130122)

THROUGH:

GOVERNMENT POLYTECHNIC RATNAGIRI

THE ACADEMIC YEAR: 2024-2025

MAHARASHTRA STATE BOARD OF TEHNICAL EDUCATION

CERTIFICATION

This is to certify that Mr. Saheem Sameer Nakhwa (2200130075), Mr.Shwet Rohit Kadam (2200130095), Mr. Vedant Vaibhav Surve (2200130097) and Mr. Viraj Satish Shinde (2200130122) from Government Polytechnic, Ratnagiri (0013) institute has completed project of final year having title Jarvis The AI Assistant during the academic year 2024-2025.the project completed in group consisting of 4 people under the guidance of the faculty guide "G. D. Patne".

G.D. PATANE

(Guide Teacher)

A.A. KARKARE

(Head of Department)

External

(Principal)

ACKNOWLEDGEMENT

It is with great pleasure and gratitude that we present this project report on "Jarvis The AI Assistant." Throughout the journey of this project, we have received invaluable support and guidance from several individuals, without whom this accomplishment would not have been possible.

We wish to convey our sincere thanks and gratitude to Ms Shruti Joshi, Ms Gauri Patne and Ms Asmita Karkare who permitted us for this project.

I would like to extend my heartfelt thanks to Mrs. Gauri Patne, for her continuous encouragement and insightful suggestions that have greatly influenced the direction and progress of this project. Her keen interest and involvement in our work have been a constant source of motivation.

We also thankful to our friends who also help us in this project directly and/or indirectly.

Finally, we are also thankful to our parents to their blessing with us with success.

This project would not have been possible without the assistance,
patience, and encouragement of everyone mentioned above.

Thank you all for being a part of this journey.

ABSTRACT

Jarvis: The Lab AI Assistant is an interactive voice-controlled AI assistant designed to enhance productivity in a lab environment. It provides features such as task management, automated responses, voice commands, and AI-based question-answering. The project integrates Python, PyQt5 for GUI, pyttsx3 for text-to-speech, Google Gemini AI for dynamic responses, and various APIs for functionalities like email, calendar, and news updates. This system ensures faster access to information and automates various lab activities.

In addition to voice recognition, Jarvis provides intelligent response generation by leveraging Google Gemini AI, allowing it to understand context and generate more relevant answers. Unlike traditional assistants, it offers a graphical user interface (GUI) that displays real-time responses, improving user interaction and accessibility. The system can also execute commands like opening applications, setting reminders, and retrieving information from Wikipedia and Google, making it a versatile tool for lab users.
Furthermore, Jarvis prioritizes efficiency and accessibility by enabling hands-free operations. It enhances user convenience by integrating speech-to-text technology, eliminating the need for manual input. The assistant can be expanded to support additional functionalities such as IoT integration for controlling lab equipment, cloud storage for data management, and multi-language support for a broader user base. With its adaptability and potential for future development, Jarvis stands as an essential AI-powered assistant for modern laboratory environments.

Another significant aspect of Jarvis is its ability to learn from user interactions and improve over time. By analyzing frequently used commands and past conversations, the system can offer personalized suggestions and anticipate user needs, making it even more efficient. This adaptability ensures that Jarvis remains relevant and continues to enhance the workflow of laboratory professionals. As AI technology advances, Jarvis has the potential to evolve into a fully autonomous virtual assistant capable of handling more complex tasks, further bridging the gap between human intelligence and machine automation.

Keywords: Ptyhon , Gemini AI , accessibility , interactions , autonomous

CONTENT

SR.NO	CONTENTS	PAGE NO.
1.	Chapter 1: Introduction and its Importance	
2.	Chapter 2: Literature Survey	
3.	Chapter 3: Scope of the Project	
4.	Chapter 4: Methodology	
5.	Chapter 5: Details of Components and their Working	
6.	Chapter 6: Code	
7	Chapter 7: Conclusion and future Scope	
8	Chapter 8: References and Bibliography	

Chapter 1 - Introduction And Background

1.1 Introduction

Jarvis: The Lab AI Assistant is designed to provide voice-controlled automation in a laboratory setup. It uses speech recognition, AI-based question answering, GUI integration, and task execution automation. The assistant simplifies access to information and executes various commands using natural language processing.

- In which the items kept within will not only be guaranteed security
- Guarantee of accounting for the duration occupation of the lockers and a subsequent charge.

1.2 **Importance**

- Automation: Reduces the need for manual operations.
- Efficiency: Provides quick responses and organizes tasks effectively.
- User-Friendly: Simple and interactive interface for ease of use.
- Multi-Functionality: Supports email, calendar, reminders, search functions, and more.
- Hands-Free Operation: Users can give commands without needing to type, making it more accessible.
- Personalized Experience: The system can remember user preferences and provide customized responses.

1.3 **Objectives of the Project**

- To create an interactive AI assistant that provides quick responses to user queries.
- To integrate voice recognition for hands-free interaction.
- To develop a GUI-based assistant for a user-friendly experience.
- To provide automation of daily tasks such as reminders, web searches, and email handling.

1.4 **Challenges in Development**

- Speech Recognition Accuracy: Background noise can affect voice command interpretation.
- Natural Language Processing: Ensuring the AI understands complex queries.
- Data Privacy and Security: Handling sensitive information securely.
- System Resource Management: Optimizing performance for smooth operation on different devices.

Chapter 2: Literature Survey, Problem Statement and Identification

2.1 Literature Survey

Paper 1:

"A Comparative Study of Virtual Assistants: Google Assistant, Siri, and Alexa"

1. **Author:**
   Prof. Amit Kumar, Dr. Rajesh Gupta, Sneha Sharma
2. **Publication:**
   International Journal of Computer Science and Information Technology (IJCSIT)
3. **Issue**
   Date: September 2021
4. **Summary:**
   This research paper evaluates the effectiveness of modern virtual assistants, including Google Assistant, Siri, and Alexa, in understanding and responding to user queries. It highlights advancements in Natural Language Processing (NLP), Machine Learning (ML), and Voice Recognition Technology used in these systems.
   The study compares these assistants based on response accuracy, contextual awareness, and user satisfaction. It discusses the role of cloud computing and AI models in enhancing their performance. The paper concludes that Google Assistant leads in accuracy and adaptability, making it ideal for AI-driven automation. The research provides valuable insights into the development of AI-based virtual assistants, such as Jarvis, and how they can be improved with better NLP models and user personalization.

Title of the Research Paper:

"Enhancing Human-Computer Interaction Through AI-Based Virtual Assistants"

1. **Author**:
   Dr. Meera Sharma, Karthik Nair, Anjali Singh
2. **Publication:**
   International Journal of Artificial Intelligence and Applications (IJAIA), Volume 14, Issue 2

3. **Issue**
   Date: March 2022
4. **Summary:**
   The paper discusses how AI-powered virtual assistants improve human-computer interaction (HCI) by allowing users to interact with devices using natural speech. It explains how speech-to-text conversion, deep learning, and generative AI models enable virtual assistants to provide real-time and context-aware responses.
   It also highlights the integration of voice assistants with smart devices and IoT systems, showcasing their potential in laboratory environments, corporate offices, and home automation. The research suggests that future AI assistants should incorporate better contextual memory and real-time adaptability, which aligns with the goals of Jarvis: The Lab AI Assistant.

Title of the Research Paper:
"Integration of AI Assistants in Laboratory Environments: A Case Study"

1. **Authors:**
   Dr. Priya Desai, Aniket Sharma, Rohit Verma
2. **Publication:**
   International Research Journal of Engineering and Technology (IRJET), Volume 09, Issue 04
3. **Issue**
   Date: April 2023
4. **Summary:**
   This research paper focuses on the implementation of AI-powered assistants in laboratory environments. It examines how voice-controlled AI assistants improve task automation, data retrieval, and equipment management.
   The study provides case studies on how AI assistants are used for automating repetitive tasks, providing quick access to research materials, and enhancing collaboration among researchers. It emphasizes that AI assistants like Jarvis can significantly reduce manual workload, allowing users to focus on more complex scientific work.

2.2 Problem Statement

1. Problem Statement for the Project

In modern laboratories and academic environments, researchers and students often face challenges in managing tasks efficiently. Searching for information, setting reminders, and performing repetitive tasks manually consume valuable time. Traditional methods of interacting with computers, such as typing commands, can be time-consuming and inefficient, especially in high-paced lab environments.

Despite advancements in AI, most available assistants like Google Assistant and Siri are designed for general use and lack customization for lab-specific tasks. This creates the need for a dedicated AI assistant that can enhance productivity by providing voice-controlled automation, task scheduling, and instant access to information within a laboratory setting.

Specific Problems Addressed

- Time-Consuming Task Execution:
  Researchers and students waste time performing manual searches, opening applications, and managing tasks instead of focusing on research.
- Lack of Personalized AI Assistants for Labs:
  Existing AI assistants are not customized for laboratory use, making them less effective for academic and research purposes.
- Limited Voice-Based Automation:
  Many laboratory processes require hands-free interaction, but most AI tools rely on manual inputs through keyboards and touchscreens.
- Difficulty in Accessing Information Quickly:
  Searching for research papers, lab manuals, or scientific data requires opening multiple tabs and navigating through various sources, reducing efficiency.
- No Integration with Productivity Tools:
  AI assistants do not integrate with academic calendars, reminders, or task scheduling tools, leading to missed deadlines and inefficient task management.

2.3 Finalized Solution

To address the challenges mentioned above, Jarvis: The Lab AI Assistant is designed as an intelligent, voice-controlled assistant tailored for laboratory environments. It provides real-time task automation, voice-activated operations, and AI-powered responses, making lab work more efficient and accessible.

Key Features and Benefits:

- Hands-Free Operation:
  Users can interact with Jarvis using voice commands, eliminating the need for manual inputs.
- AI-Powered Assistance:
  The system leverages Google Gemini AI to provide accurate, context-aware responses to research queries.
- Task Automation:
  Jarvis can set reminders, schedule tasks, and send notifications, ensuring that no important task is overlooked.
- Real-Time Information Retrieval:
  The assistant can search research papers, summarize documents, and provide quick access to scientific data from sources like Wikipedia and Google.
- Customizable and Expandable:
  Jarvis is built to adapt to user needs by integrating additional APIs for IoT control, cloud storage, and lab equipment management.
- Enhanced Security and Privacy:
  The system ensures secure processing of user data, preventing unauthorized access to sensitive lab information.
- Integration with Academic Tools:
  Jarvis can sync with Google Calendar, Google Drive, and email services, allowing users to manage their schedules seamlessly.

By implementing Jarvis: The Lab AI Assistant, laboratories and academic institutions can create a more productive, organized, and technology-driven workspace that enhances efficiency while reducing manual workload.

Chapter 3- Scope of the project

‘Jarvis: The AI Assistant’

Jarvis: The Lab AI Assistant provides voice-controlled automation beyond traditional AI assistants, specifically designed for laboratory and academic environments. In labs, students and researchers need quick access to information, efficient task management, and hands-free interaction while working on complex tasks. Jarvis integrates speech recognition, AI-based dynamic responses, GUI interaction, and task execution automation, making lab operations more efficient and interactive.

This system offers features like real-time response generation, voice-command execution, task scheduling, personalized assistance, and integration with web services. Most importantly, it eliminates manual effort, helping users focus on research and experimentation rather than repetitive tasks.
Thus, the scope of our project will include:

3.1 Core Functionalities

1. **Speech Recognition Module:**
    - Uses speech_recognition to convert spoken words into text.
    - Allows hands-free operation for executing tasks efficiently.
2. **AI-Based Answering System:**
    - Utilizes Google Gemini API to generate intelligent, real-time responses.
    - Provides context-aware answers for research queries, helping students and professionals.
3. **Task Automation:**
    - Schedules reminders, sets alarms, and creates calendar events.
    - Automates email sending, internet searches, and file management using voice commands.
4. **GUI Interface (Graphical User Interface):**

- Built using **PyQt5**, providing a **user-friendly and visually interactive experience**.
- Displays responses, reminders, and task notifications in an intuitive manner.

5. **Integration with External Services:**
   - Fetches Wikipedia summaries, YouTube videos, and Google search results for user queries.
   - Helps users search for research papers, get current news, and access study materials efficiently.
6. **Security and Privacy Protection:**
   - Logs user interactions for analysis and improvement.
   - Ensures **data security by limiting access to sensitive information.**

3.2 Software-Related Components

Jarvis is developed using Python and integrates multiple libraries and APIs for efficient voice recognition, AI-powered response generation, and GUI interaction. Each module is separately coded and then integrated into a single, optimized system.

Key Software Components:

- **Python 3+** – The core programming language used for AI and GUI development.
- **PyQt5** – Used for designing the graphical user interface (GUI).
- **Speech Recognition** – Converts voice input into text for processing.
- **Pyttsx3** – Provides text-to-speech conversion for AI-generated responses.
- **Google Gemini API** – Enables AI-powered dynamic responses to user queries.
- **Wikipedia & Web APIs** – Fetches real-time information from the internet.

3.3 Key Functional Areas of Jarvis

1. **Voice Command Execution:**

- Users can operate the system **completely hands-free** using speech commands.
- Recognizes and processes multiple types of commands efficiently.

2. **AI-Based Question Answering:**
   - Uses Google Gemini AI to answer general queries, summarize texts, and provide insights.
   - Can be expanded to support domain-specific knowledge for specialized research.
3. **Task Scheduling and Productivity Management:**
   - Supports reminders, countdown timers, and meeting schedules.
   - Can integrate with Google Calendar for better task management.
4. **Personalized Assistance:**
   - Stores frequently used commands and user preferences for customized interactions.
   - Can be adapted to individual needs in academic and professional settings.

Security and Privacy Considerations

- **User Data Protection:** Ensures secure handling of voice commands and prevents unauthorized access to stored data.
- **Command Restriction:** Limits AI's capabilities to prevent unintended or harmful commands.
- **Log Management:** Maintains records of past interactions to improve AI accuracy while maintaining user privacy.

Customization and Future Expansions

The scope of Jarvis: The Lab AI Assistant is highly extensible and can be customized for various applications. The system can evolve further with additional features, including:

1. **Integration with IoT Devices:**
   - Controlling smart lab equipment using voice commands.

- Automating lab lights, fans, and other electronic devices with AI assistance.

2. **Advanced Machine Learning Features:**
    - AI-powered context-aware conversation memory for personalized interactions.
    - Predicting user needs based on past commands and frequently asked queries.
3. **Cloud-Based AI Processing:**
    - Storing and syncing user preferences across multiple devices.
    - Enhancing AI performance by integrating cloud-based machine learning models.
4. **Multi-Language Support:**
    - Expanding support to different languages for broader accessibility.
    - Enabling real-time language translation using AI.
5. **Collaboration with Learning Platforms:**
    - Integration with Google Scholar, ResearchGate, and other academic resources for easy information retrieval.
    - AI-assisted summarization of complex research papers and technical documents.

Summary

The scope of Jarvis: The Lab AI Assistant is extensive and adaptable, making it suitable for a variety of laboratory, academic, and professional environments. The assistant improves efficiency by automating repetitive tasks, providing quick information access, and integrating voice-based controls into daily workflows.

With continuous enhancements, Jarvis can evolve into a highly advanced AI system, revolutionizing the way students, researchers, and professionals interact with technology in their respective fields.

Chapter 4: Methodology

‘Jarvis: The AI Assistant’

Our team set out to develop Jarvis: The Lab AI Assistant, an intelligent voice-controlled system designed to enhance efficiency and productivity in laboratory environments. The assistant provides hands-free interaction for students and researchers by automating tasks, retrieving information, and integrating AI-powered responses.

Jarvis is designed to simplify user interactions with technology, reducing the need for manual searches, typing commands, or navigating applications manually. The system is equipped with speech recognition, AI-driven responses, and task execution capabilities, making it an essential tool for modern labs.

When a user initiates Jarvis, it listens for voice commands and responds accordingly. If a user requests information, the AI assistant will fetch real-time answers from sources like Google or Wikipedia. For task automation, Jarvis can set reminders, send emails, schedule meetings, and manage files using voice commands. Additionally, the system provides a graphical user interface (GUI) where users can view responses, access logs, and manage settings.

By integrating Google Gemini AI for intelligent responses and PyQt5 for GUI interaction, Jarvis ensures an efficient and user-friendly experience. The system is designed to work seamlessly in a laboratory environment, assisting students and researchers by providing quick access to research materials, organizing tasks, and answering queries instantly.

System Workflow

1. **Voice Command Activation:**
    - The user speaks a command into the microphone.
    - The system detects the voice input and converts it into text using Speech Recognition.
2. **Processing the Request:**
    - The Google Gemini AI API processes the user’s query and generates an

appropriate response.

- For task execution commands (e.g., setting reminders, sending emails), the system activates the required module.

3. **Displaying the Response:**
   - The response is converted into speech using pyttsx3 and played back to the user.
   - The GUI interface displays the response on the screen for better readability.
4. **Executing Automation Tasks:**
   - If a user requests a task to be performed, such as opening an application or searching for a file, Jarvis executes the command using system automation tools.
   - For reminders and calendar events, Jarvis integrates with Google Calendar and notification systems.
5. **Data Logging and Interaction History:**
   - The system maintains a **log of user interactions** for improving future responses.
   - Users can view their command history via the GUI.

Key Functionalities of the System

- **Speech-to-Text Conversion:**
  - Converts spoken commands into text using Speech Recognition.
- **AI-Based Response Generation:**
  - Uses Google Gemini API to provide intelligent and context-aware responses.
- **Graphical User Interface (GUI):**
  - Provides visual interaction with the assistant, displaying responses and task details.
- **Task Execution and Automation:**

    - Handles email management, reminders, web searches, and file operations.

- **Security and User Privacy:**
    - Ensures secure processing of user queries and protects stored data.

System Benefits

1. **Hands-Free Operation:**
    - Enables users to interact with technology without using a keyboard or mouse.
2. **Real-Time Information Retrieval:**
    - Fetches data instantly from the web, reducing time spent on research.
3. **Multi-Tasking Capabilities:**
    - Can perform multiple operations simultaneously, such as setting reminders while searching for documents.
4. **User-Friendly Interface:**
    - Provides a simple and intuitive GUI for easy navigation and interaction.
5. **Automation of Repetitive Tasks:**
    - Reduces the need for manual effort in setting reminders, checking emails, and scheduling tasks.

Security Measures and Data Protection

- **Access Control:**
    - Prevents unauthorized access to AI-generated logs and user preferences.
- **Secure API Communication:**
    - Ensures encrypted data transmission between the assistant and web services.
- **Privacy Protection:**
    - Limits data retention to prevent the storage of unnecessary personal

information.

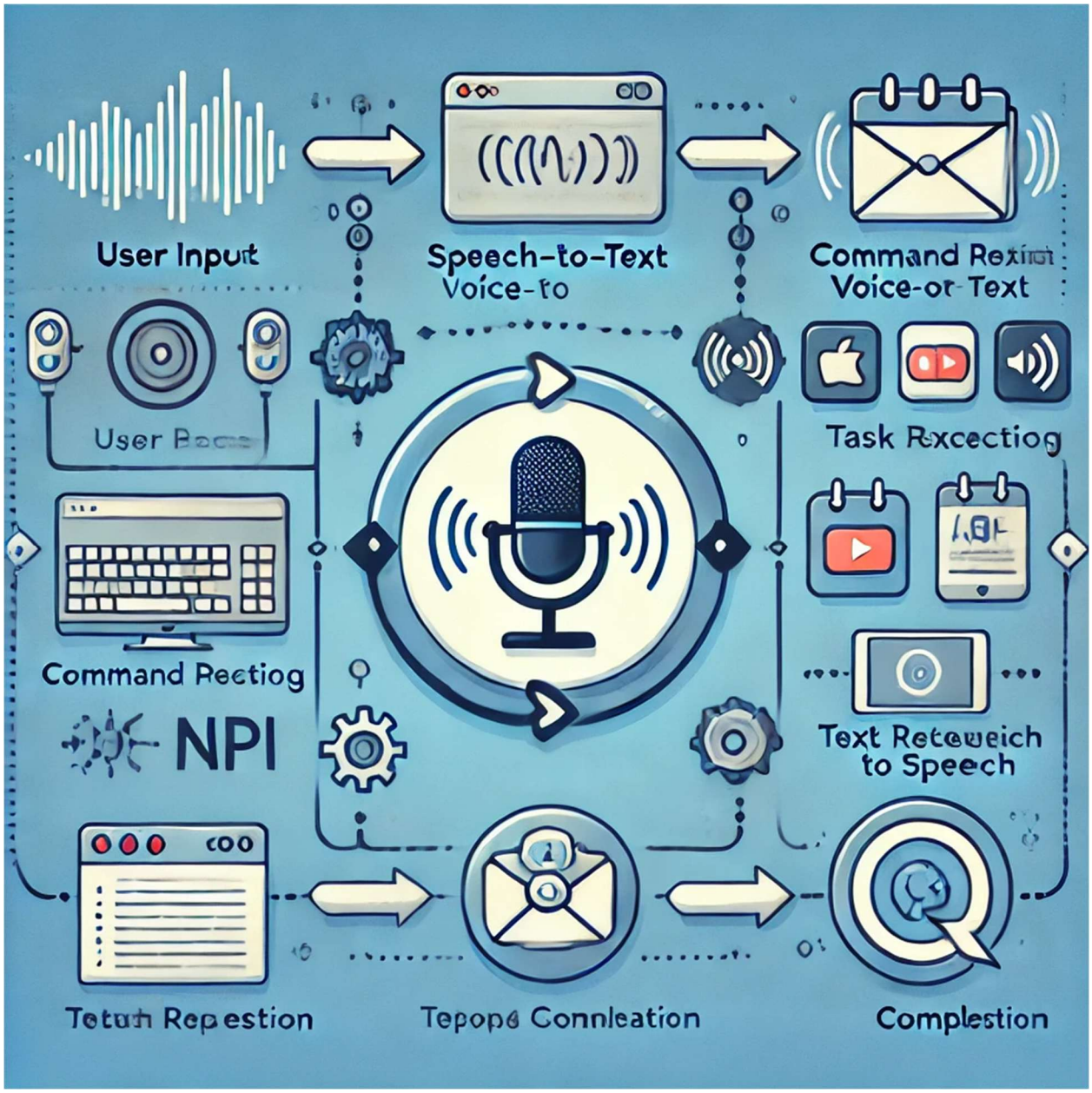
Future Enhancements

- **Integration with IoT Devices:**
  - Allowing Jarvis to control smart lab equipment through voice commands.
- **Advanced AI Features:**
  - Machine learning algorithms to improve response accuracy over time.
- **Cloud Synchronization:**
  - Enabling users to access their settings and logs from multiple devices.
- **Multi-Language Support:**
  - Expanding capabilities to support multiple languages for broader accessibility.

Summary

Jarvis: The Lab AI Assistant is designed to enhance efficiency, improve accessibility, and automate various tasks in laboratory environments. With AI-driven responses, speech recognition, and task automation, Jarvis helps users save time and increase productivity.

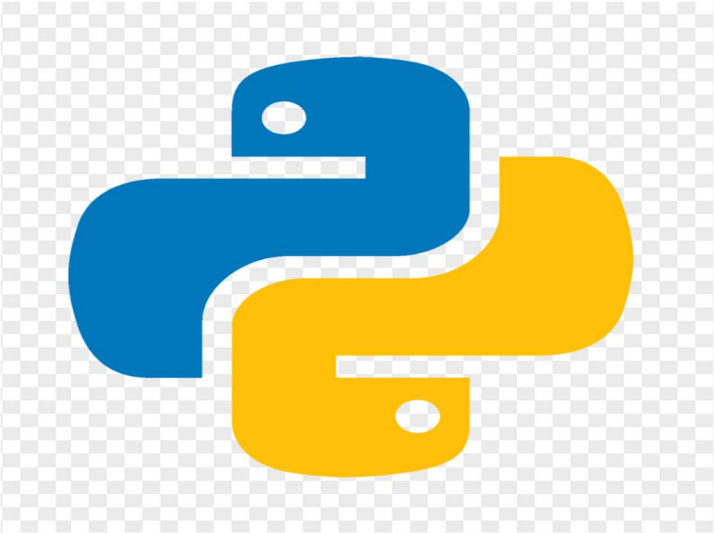
The system ensures seamless operation, high security, and easy scalability, making it a valuable tool for students, researchers, and professionals in need of a smart, voice-controlled AI assistant.


User Input
Speech-to-Text
Voice-to
Command Rexiot
Voice-or-Text
User Bocs
Task Rxcectiog
Command Reetiog
NPI
Text Retcweich
to Speech
Tetuth Repestion
Tepopo Connleation
Complestion

Chapter 5: Details of design, working and Processes

The System Implementation:

Python:



Python is a powerful high-level, interpreted programming language widely used in AI and automation projects. Its object-oriented and functional programming capabilities make it an ideal choice for developing Jarvis: The Lab AI Assistant. Python's rich library ecosystem and cross-platform support allow seamless integration with speech recognition, AI models, GUI applications, and automation tasks.

Key features of Python used in this project:

- **Speech Recognition:** Converts spoken words into text for AI processing.
- **Pyttsx3:** Provides text-to-speech capabilities for AI-generated responses.
- **Google Gemini API:** Enables real-time AI-based responses.
- **PyQt5:** Used for building an intuitive Graphical User Interface (GUI).
- **Web browser & Wikipedia API:** Helps fetch online data and answer user queries.

Since **Python supports multiprocessing and multithreading**, it allows **Jarvis to handle multiple user requests simultaneously**, making it efficient and responsive.

Components required:

Sr no.	Component Name	Quantity
1	Microphone (for voice input)	1
2	Speaker (for AI responses)	1
3	Computer/Laptop with Python	1
4	Internet Connection (for AI)	1

System Design & Working

Introduction to the AI Assistant System

Jarvis: The Lab AI Assistant is a voice-activated AI system designed to enhance productivity in laboratory environments by allowing hands-free interaction. It processes user commands using speech recognition, generates responses through Google Gemini AI, and provides a user-friendly GUI for interaction.

Working of the System

1. **Voice Recognition and Command Processing:**
    - The user speaks a command into the microphone.
    - SpeechRecognition converts the spoken words into text.
    - The command is processed to determine the required action.
2. **AI Response Generation:**
    - If the command requires an answer or information retrieval, Jarvis queries the Google Gemini API.
    - The AI processes the request and generates a context-aware response.
3. **Task Execution & Automation:**
    - For commands like "set a reminder" or "send an email," the system executes task automation functions.
    - For file management tasks, the system interacts with the operating system to perform actions like opening, deleting, or moving files.
4. **Graphical User Interface (GUI):**
    - The response is displayed on the GUI, along with logs of previous interactions.
    - Users can manually input commands if they prefer text-based interaction.
5. **Voice Output:**
    - The system uses pyttsx3 to convert text-based responses into speech, ensuring an interactive user experience.

Key Hardware and Software Modules

1. Speech Recognition Module

- Converts voice commands into text using Google's speech recognition API.
- Enables hands-free interaction, allowing users to control the assistant via

voice.

2. AI-Based Answering System

- Uses Google Gemini API for intelligent, real-time responses.
- Fetches relevant answers to technical queries, general knowledge questions, and lab-related topics.

3. GUI Interface (Graphical User Interface)

- Developed using PyQt5, providing a user-friendly visual representation of responses.
- Allows users to interact with Jarvis through a text box or voice input button.

4. Task Automation Module

- Supports automated email sending, reminder setting, web searches, and calendar event creation.
- Performs system-based operations like opening software applications.

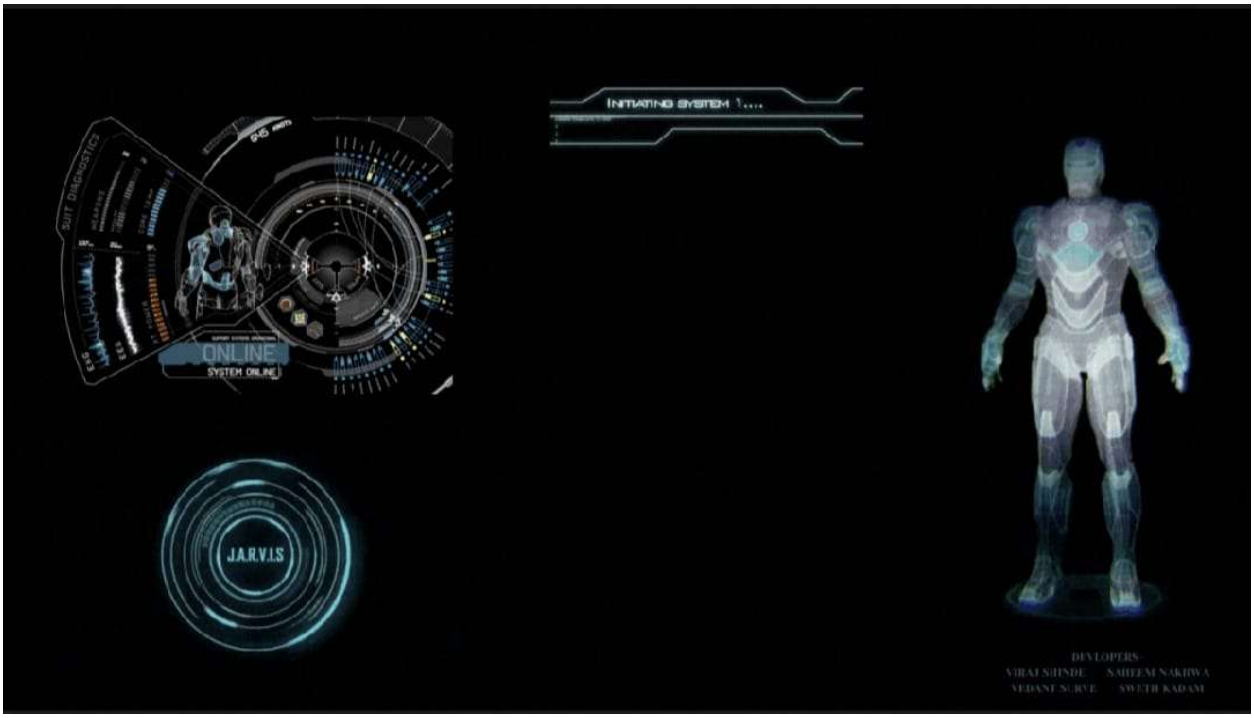
5. Security & Privacy Protection

- Stores interaction logs securely, ensuring user privacy.
- Prevents unauthorized access to sensitive data.

Why We Chose Python for This Project?

1. **AI & Machine Learning Integration:**
    - Python supports AI frameworks like TensorFlow, OpenAI GPT, and Google Gemini AI, making it ideal for AI-driven virtual assistants.
2. **Wide Library Support:**
    - Built-in support for speech recognition, text-to-speech, and GUI development.
3. **Cross-Platform Compatibility:**
    - The assistant runs on Windows, Linux, and macOS without major modifications.

4. **Efficient Performance:**
    - Python handles multithreading and parallel processing, enabling real-time responses.



Application in Our Project

1. Speech-to-Text Processing:

```
import speech_recognition as sr


def command():
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print("Listening...")
        audio = r.listen(source)
```

```
    try:
        return r.recognize_google(audio, language='en-in')
    except:
        return "I couldn't understand that."
```

2. **AI-Based Response Generation Using Google Gemini API:**

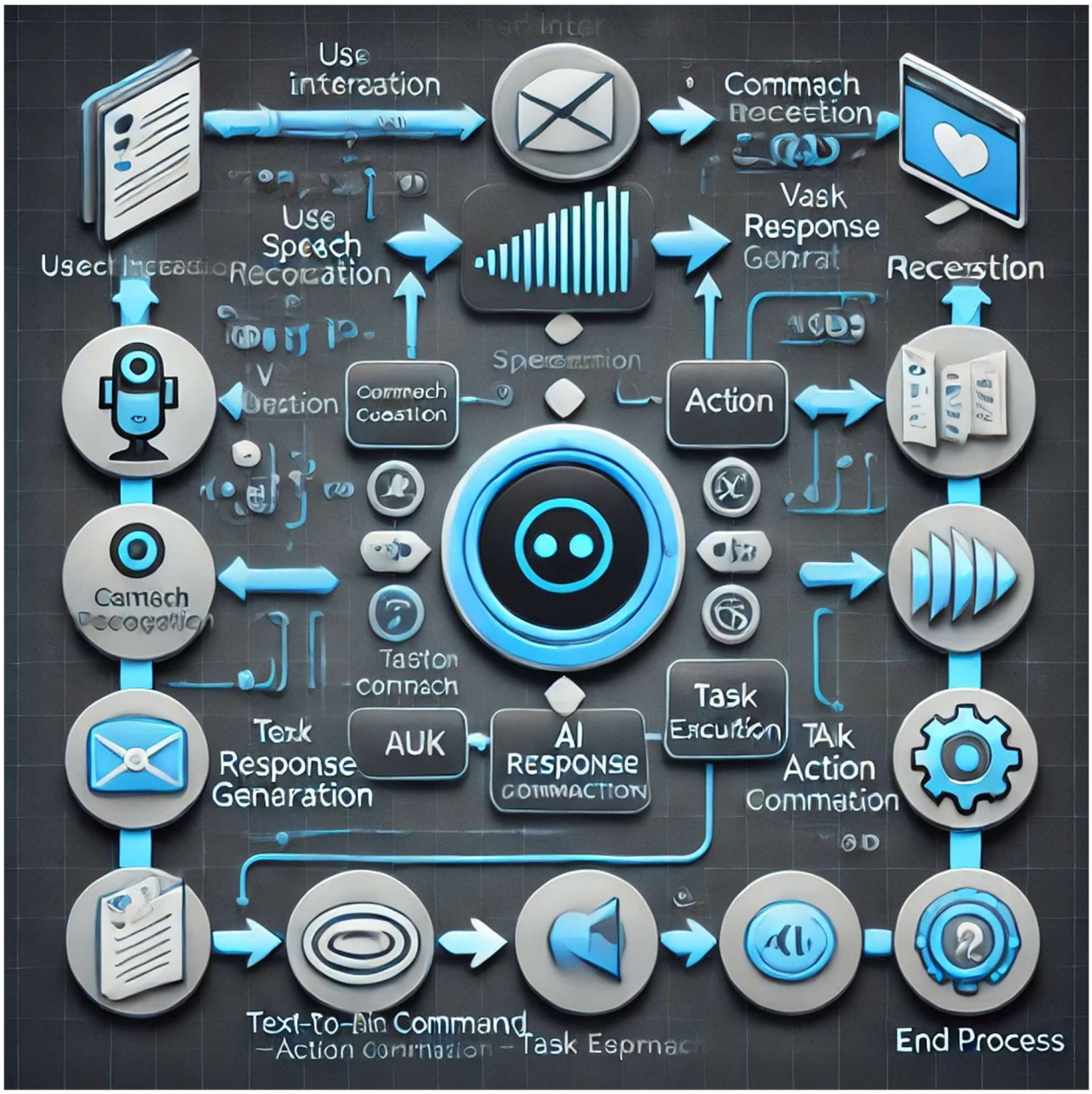
```
import google.generativeai as genai

genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel('gemini-1.5-flash')

def gemini_response(query):
    response = model.generate_content(query)
    return response.text
```

3. **GUI Implementation Using PyQt5:**

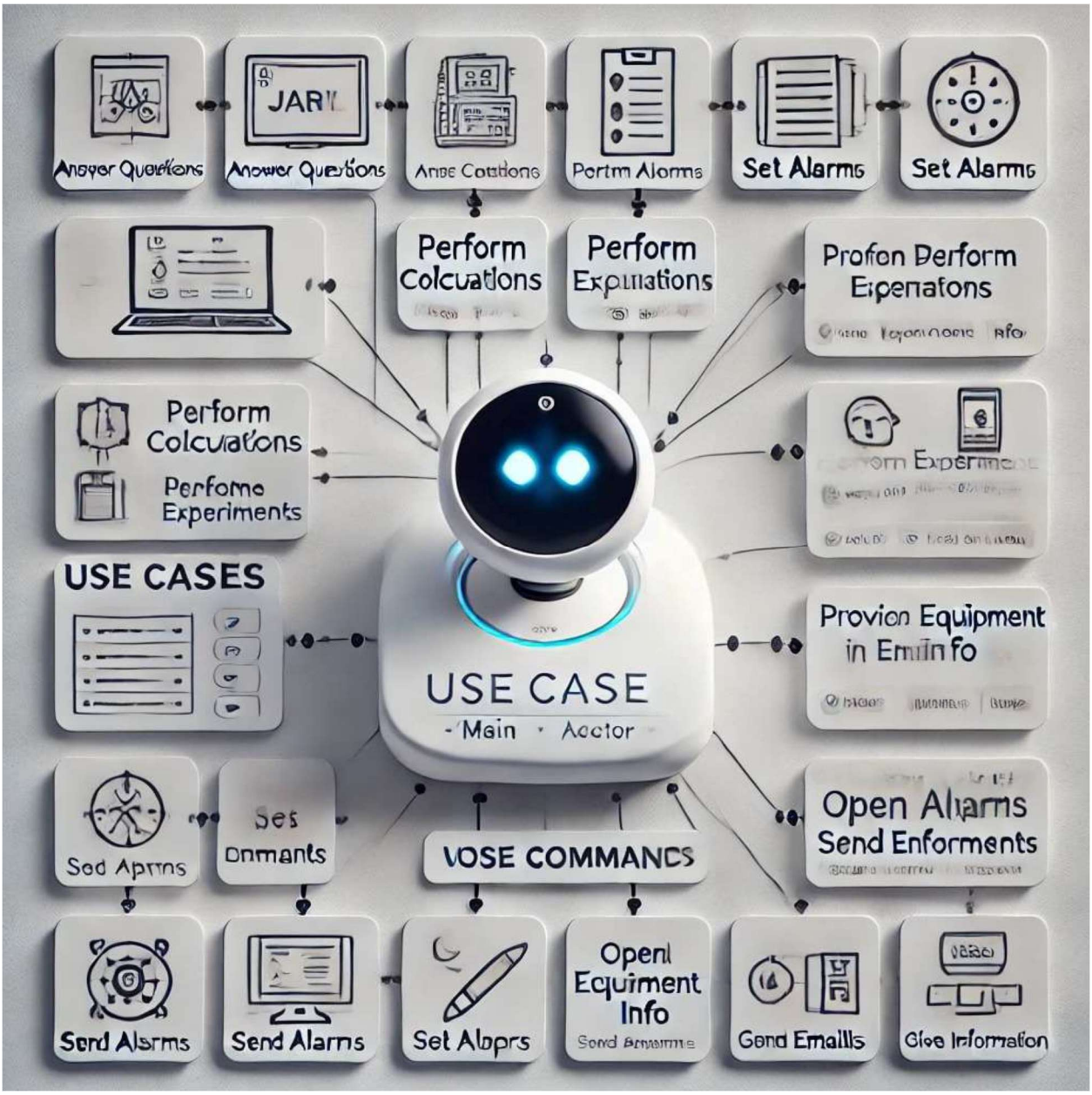
```
from PyQt5 import QtWidgets
from PyQt5.QtWidgets import QApplication, QMainWindow
import sys

def window():
    app = QApplication(sys.argv)
    win = QMainWindow()
    win.setGeometry(200, 200, 500, 300)
```

```
win.setWindowTitle("Jarvis AI Assistant")
win.show()
sys.exit(app.exec_())
window()
```

1. Activity Diagram



- Use case diagram



Summary

Jarvis: The Lab AI Assistant is a fully functional, AI-powered virtual assistant designed to enhance efficiency, automate repetitive tasks, and provide real-time AI responses. By integrating speech recognition, machine learning, and a user-friendly GUI, Jarvis helps users work faster, organize tasks, and access information effortlessly.

Chapter 6: Code

```
class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        MainWindow.setObjectName("MainWindow")
        MainWindow.resize(1317, 806)
        self.centralwidget = QtWidgets.QWidget(MainWindow)
        self.label = QtWidgets.QLabel(self.centralwidget)
        self.label.setGeometry(QtCore.QRect(0, -60, 1341, 851))

        # Use QMovie to display animated GIF
        self.movie = QtGui.QMovie("gui.gif")
        self.label.setMovie(self.movie)
        self.movie.start()

        self.label.setScaledContents(True)
        self.textBrowser_2 = QtWidgets.QTextBrowser(self.centralwidget)
        self.textBrowser_2.setGeometry(QtCore.QRect(530, 251, 471, 361))
        self.pushButton_2 = QtWidgets.QPushButton("RUN", self.centralwidget)
        self.textBrowser_2.setStyleSheet("background: transparent; color: #00FF00;
font-size: 16px; border: none;")
        self.pushButton_2.setGeometry(QtCore.QRect(530, 640, 101, 31))
        self.pushButton_2.setStyleSheet("""
            QPushButton {
                background-color: #28a745;
```

```
        color: white;
        font-size: 16px;
        font-weight: bold;
        border-radius: 10px;
        border: 2px solid #218838;
    }
    QPushButton:hover {
        background-color: #218838;
    }
    QPushButton:pressed {
        background-color: #1e7e34;
    }
""")
self.pushButton_3 = QtWidgets.QPushButton("STOP", self.centralwidget)
self.pushButton_3.setGeometry(QtCore.QRect(900, 640, 101, 31))
self.pushButton_3.setStyleSheet("""
    QPushButton {
        background-color: #dc3545;
        color: white;
        font-size: 16px;
        font-weight: bold;
        border-radius: 10px;
        border: 2px solid #c82333;
    }
```

```
            QPushButton:hover {
                background-color: #c82333;
            }
            QPushButton:pressed {
                background-color: #a71d2a;
            }
        """)
        self.textBrowser_3 = QtWidgets.QTextBrowser(self.centralwidget)
        self.textBrowser_3.setGeometry(QtCore.QRect(140, 10, 191, 31))
        self.textBrowser_3.setStyleSheet("background:transparent; border: none; color: white; font-size: 14px;")
        self.textBrowser_4 = QtWidgets.QTextBrowser(self.centralwidget)
        self.textBrowser_4.setGeometry(QtCore.QRect(1070, 10, 191, 31))
        self.textBrowser_4.setStyleSheet("background:transparent; border: none; color: white; font-size: 14px;")
        MainWindow.setCentralWidget(self.centralwidget)
        self.pushButton_2.clicked.connect(self.start_jarvis)
        self.pushButton_3.clicked.connect(self.stop_jarvis)
        self.timer = QtCore.QTimer()
        self.timer.timeout.connect(self.update_time)
        self.timer.start(1000)
        self.jarvis_thread = None


    def update_time(self):
        self.textBrowser_3.setText(datetime.now().strftime("%H:%M:%S"))
```

```
        self.textBrowser_4.setText(datetime.now().strftime("%d-%m-%Y"))

    def start_jarvis(self):
        self.textBrowser_2.append("Jarvis started...")
        self.jarvis_thread = JarvisThread()
        self.jarvis_thread.update_signal.connect(self.textBrowser_2.append)
        self.jarvis_thread.start()

    def stop_jarvis(self):
        if self.jarvis_thread:
            self.jarvis_thread.terminate()
        speak("Shutting down")
        QtWidgets.qApp.quit()

class MovableMainWindow(QtWidgets.QMainWindow):
    def __init__(self):
        super(MovableMainWindow, self).__init__()
        self.oldPos = None

    def mousePressEvent(self, event):
        self.oldPos = event.globalPos()

    def mouseMoveEvent(self, event):
        if self.oldPos:
```

```
        delta = QtCore.QPoint(event.globalPos() - self.oldPos)
        self.move(self.x() + delta.x(), self.y() + delta.y())
        self.oldPos = event.globalPos()

class FaceRecognitionThread(QThread):
    update_frame = QtCore.pyqtSignal(QImage)
    authentication_result = QtCore.pyqtSignal(bool)

    def __init__(self, reference_image_path):
        super().__init__()
        self.reference_image_path = reference_image_path
        self.running = True

    def run(self):
        # Load face cascade for simple face detection - no dlib required
        try:
                face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

        # Access webcam (use camera index 1 for DroidCam)
        cap = cv2.VideoCapture(0)  # Changed from 0 to 1 for DroidCam
        if not cap.isOpened():
            print("Cannot open camera")
            self.authentication_result.emit(False)
            return
```

```
face_detected_frames = 0
required_frames = 20  # Need to detect face in 20 frames to authenticate
attempts = 0
max_attempts = 100  # About 10 seconds of trying

while self.running and attempts < max_attempts:
    ret, frame = cap.read()
    if not ret:
        print("Can't receive frame. Exiting...")
        break

    # Convert to grayscale for face detection
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, 1.3, 5)

    # Add visual effects to frame
    # Add a border
    frame = cv2.copyMakeBorder(frame, 10, 10, 10, 10, cv2.BORDER_CONSTANT, value=[0, 120, 255])

    # Add scanning effect overlay
    overlay = frame.copy()
```

```
        # Draw horizontal scan line
        scan_position = (attempts % 50) * (frame.shape[0] // 50)
         cv2.line(overlay, (0, scan_position), (frame.shape[1], scan_position), (0,
255, 0), 2)
        frame = cv2.addWeighted(overlay, 0.7, frame, 0.3, 0)

        # Add system status text
         cv2.putText(frame, "JARVIS SECURITY SYSTEM", (frame.shape[1]//2
- 150, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

        if len(faces) > 0:
            # Face detected
            face_detected_frames += 1

            # Draw rectangle around face
            for (x, y, w, h) in faces:
                # Draw fancy rectangle
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

                # Draw corner lines for futuristic look
                corner_length = 20
                # Top left
                cv2.line(frame, (x, y), (x + corner_length, y), (0, 255, 255), 2)
                cv2.line(frame, (x, y), (x, y + corner_length), (0, 255, 255), 2)
```

```
# Top right
cv2.line(frame, (x+w, y), (x+w - corner_length, y), (0, 255, 255), 2)
cv2.line(frame, (x+w, y), (x+w, y + corner_length), (0, 255, 255), 2)
# Bottom left
cv2.line(frame, (x, y+h), (x + corner_length, y+h), (0, 255, 255), 2)
cv2.line(frame, (x, y+h), (x, y+h - corner_length), (0, 255, 255), 2)
# Bottom right
cv2.line(frame, (x+w, y+h), (x+w - corner_length, y+h), (0, 255,
255), 2)
cv2.line(frame, (x+w, y+h), (x+w, y+h - corner_length), (0, 255,
255), 2)

progress = min(face_detected_frames / required_frames, 1.0)
bar_width = int(w * progress)

# Draw progress bar
cv2.rectangle(frame, (x, y+h+5), (x+bar_width, y+h+15), (0, 255,
0), -1)
cv2.rectangle(frame, (x, y+h+5), (x+w, y+h+15), (255, 255, 255),
1)

# Progress percentage
percentage = int(progress * 100)
cv2.putText(frame, f"Scanning: {percentage}%", (x, y+h+30),
        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
```

```
# Check if we have enough consistent detections
if face_detected_frames >= required_frames:
    # Authentication successful
    # Create success overlay
    success_overlay = frame.copy()
        cv2.rectangle(success_overlay, (0, 0), (frame.shape[1], frame.shape[0]), (0, 255, 0), -1)

    # Blend with original frame
    frame = cv2.addWeighted(success_overlay, 0.3, frame, 0.7, 0)

    # Add success text
    cv2.putText(frame, "ACCESS GRANTED", (frame.shape[1]//2 - 150, frame.shape[0]//2),
        cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 3)
        cv2.putText(frame, "AUTHENTICATION SUCCESSFUL", (frame.shape[1]//2 - 200, frame.shape[0]//2 + 50),
        cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

    # Convert frame to QImage and emit
    h, w, ch = frame.shape
        qt_image = QImage(frame.data, w, h, ch * w, QImage.Format_RGB888)
    self.update_frame.emit(qt_image.rgbSwapped())

    # Wait a moment to display the success message
```

```
                time.sleep(1.5)

                # Authentication successful - speak confirmation
                speak("Authentication successful. Welcome back, sir.")

                # Authentication successful
                cap.release()
                print("Authentication successful, emitting True signal")
                self.authentication_result.emit(True)
                return
            else:
                # Reset counter if face is lost
                face_detected_frames = max(0, face_detected_frames - 1)
                # Warning text
                cv2.putText(frame, "No face detected", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
                cv2.putText(frame, "Please position your face in the camera view", (50, 80),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

            # Add scanning line effect

            # Convert frame to QImage and emit
            h, w, ch = frame.shape
            qt_image = QImage(frame.data, w, h, ch * w, QImage.Format_RGB888)
```

```
            self.update_frame.emit(qt_image.rgbSwapped())

            attempts += 1
            time.sleep(0.1)

        # If we get here without successful authentication
        cap.release()
        speak("Authentication failed. Access denied.")
        print("Authentication failed, emitting False signal")
        self.authentication_result.emit(False)
    except Exception as e:
        print(f"Error in facial recognition: {e}")
        # In case of error, still try to proceed with authentication
        speak("Authentication system error. Proceeding with startup.")
        self.authentication_result.emit(True)

def stop(self):
    self.running = False
    self.wait()

class FaceAuthenticationWindow(QtWidgets.QMainWindow):
    def __init__(self, reference_image_path="face_reference.jpg"):
        super().__init__()
        self.reference_image_path = reference_image_path
```

```
        print("Initializing FaceAuthenticationWindow")
        self.initUI()

    def initUI(self):
        self.setWindowTitle('JARVIS Security System')
        self.setGeometry(100, 100, 800, 600)
        self.setStyleSheet("background-color: #1E1E1E;")

        # Set window icon
        try:
            self.setWindowIcon(QtGui.QIcon("jarvis_icon.png"))
        except Exception as e:
            print(f"Error setting window icon: {e}")

        # Central widget
        central_widget = QtWidgets.QWidget()
        self.setCentralWidget(central_widget)

        # Main layout
        main_layout = QtWidgets.QVBoxLayout(central_widget)
        main_layout.setContentsMargins(20, 20, 20, 20)

        # Title label
        title_label = QtWidgets.QLabel("JARVIS SECURITY AUTHENTICATION")
```

```
title_label.setAlignment(QtCore.Qt.AlignCenter)
title_label.setStyleSheet("color: #00A9F4; font-size: 24px; font-weight: bold; margin-bottom: 10px;")
main_layout.addWidget(title_label)

# Video frame with border
video_frame = QtWidgets.QFrame()
video_frame.setFrameShape(QtWidgets.QFrame.StyledPanel)
video_frame.setStyleSheet("border: 2px solid #00A9F4; border-radius: 10px; padding: 5px; background-color: #2D2D30;")
video_layout = QtWidgets.QVBoxLayout(video_frame)

# Video display label
self.video_label = QtWidgets.QLabel()
self.video_label.setMinimumSize(700, 450)
self.video_label.setAlignment(QtCore.Qt.AlignCenter)
self.video_label.setStyleSheet("border: none;")
video_layout.addWidget(self.video_label)

main_layout.addWidget(video_frame)

# Status label
self.status_label = QtWidgets.QLabel("Initializing facial recognition system...")
self.status_label.setStyleSheet("font-size: 18px; color: #00A9F4; margin-top: 10px;")
```

```
        self.status_label.setAlignment(QtCore.Qt.AlignCenter)
        main_layout.addWidget(self.status_label)

        # Start face recognition with voice prompt
        print("Setting up face recognition timer")
        speak("Initializing security systems")
        QTimer.singleShot(1500, self.start_face_recognition)

    def start_face_recognition(self):
        print("Starting face recognition")
        speak("Beginning facial authentication")
        self.status_label.setText("Looking for your face...")
        # Create and start the face recognition thread
        self.face_thread = FaceRecognitionThread(self.reference_image_path)
        print("Connecting signals")
        self.face_thread.update_frame.connect(self.update_frame)
        self.face_thread.authentication_result.connect(self.handle_authentication)
        print("Starting face thread")
        self.face_thread.start()

    def update_frame(self, qt_image):
        pixmap = QPixmap.fromImage(qt_image)
                self.video_label.setPixmap(pixmap.scaled(self.video_label.width(),
self.video_label.height(), QtCore.Qt.KeepAspectRatio))
```

```
    def handle_authentication(self, success):
        print(f"Authentication result: {success}")
        if success:
            self.status_label.setText("Authentication Successful! Launching JARVIS...")
            self.status_label.setStyleSheet("font-size: 18px; color: #00FF00; margin-top: 10px;")
            # Wait a moment to display the success message
            print("Setting up launch timer")
            QTimer.singleShot(2000, self.launch_jarvis)
        else:
            self.status_label.setText("Authentication Failed! Access Denied.")
            self.status_label.setStyleSheet("font-size: 18px; color: #FF0000; margin-top: 10px;")
            # Wait a moment and then close the application
            QTimer.singleShot(3000, self.close)

    def launch_jarvis(self):
        print("Launch method called")
        try:
            # Close the authentication window
            print("Closing authentication window")
            self.close()

            # Launch Jarvis in a new process
            print("Launching Jarvis in new process")
```

```
            # Use subprocess to launch a new instance with the --gui-only flag
            subprocess.Popen([sys.executable, "endgame.py", "--gui-only"])

            # Exit this process
            print("Exiting authentication process")
            sys.exit(0)
        except Exception as e:
            print(f"Error launching Jarvis: {e}")
            with open("jarvis_error.log", "w") as file:
                file.write(f"Error launching Jarvis: {e}")

    def closeEvent(self, event):
        # Stop the face recognition thread when closing the window
        if hasattr(self, 'face_thread'):
            self.face_thread.stop()
        event.accept()

def save_reference_image():
    """Capture and save a reference image of the user's face"""
    if os.path.exists("face_reference.jpg"):
        return  # Reference image already exists

    print("Capturing reference image of your face...")
    cap = cv2.VideoCapture(0)  # Changed from 0 to 1 for DroidCam
```

```
    if not cap.isOpened():
        print("Cannot open camera")
        return False

    # Wait a bit to let the camera adjust
    for _ in range(10):
        cap.read()
        time.sleep(0.1)

    ret, frame = cap.read()
    cap.release()

    if ret:
        # Save the reference image
        cv2.imwrite("face_reference.jpg", frame)
        print("Reference image saved as 'face_reference.jpg'")
        return True
    else:
        print("Failed to capture reference image")
        return False

def launch_jarvis_standalone():
    """Standalone function to launch Jarvis directly"""
    try:
```

```
        print("Launching Jarvis standalone")
        app = QtWidgets.QApplication(sys.argv)
        MainWindow = MovableMainWindow()
        ui = Ui_MainWindow()
        ui.setupUi(MainWindow)
        MainWindow.show()
        print("Jarvis GUI should be visible now")
        sys.exit(app.exec_())
    except Exception as e:
        print(f"Error in standalone launch: {e}")
        with open("standalone_error.log", "w") as f:
            f.write(f"Standalone launch error: {e}")

if __name__ == "__main__":
    # Check if --gui-only flag is set (just launch the main GUI directly)
    if len(sys.argv) > 1 and sys.argv[1] == "--gui-only":
        print("Running in GUI-only mode - bypassing authentication")
        launch_jarvis_standalone()
        sys.exit(0)

    try:
        # Check if running in test mode (without camera)
        if len(sys.argv) > 1 and sys.argv[1] == "--no-camera":
            print("Running in test mode - bypassing face authentication")
```

```
        # Launch Jarvis directly
        launch_jarvis_standalone()
        sys.exit(0)

    # Check if reference image exists, otherwise create one
    if not os.path.exists("face_reference.jpg"):
        # If no reference image, capture one
        app = QtWidgets.QApplication(sys.argv)
        msgBox = QtWidgets.QMessageBox()
        msgBox.setWindowTitle("First Time Setup")
        msgBox.setText("No reference face image found. Would you like to capture one now? (Or run with --no-camera flag to bypass)")
                        msgBox.setStandardButtons(QtWidgets.QMessageBox.Yes | QtWidgets.QMessageBox.No)
        msgBox.setDefaultButton(QtWidgets.QMessageBox.Yes)

        if msgBox.exec_() == QtWidgets.QMessageBox.Yes:
            save_reference_image()
        else:
            sys.exit()

    # Start the authentication app
    try:
        app = QtWidgets.QApplication(sys.argv)
        window = FaceAuthenticationWindow()
```

```
        window.show()
        sys.exit(app.exec_())
    except Exception as auth_error:
        print(f"Error in face authentication window: {auth_error}")
        with open("auth_error.log", "w") as f:
            f.write(f"Authentication window error: {auth_error}")

        # Fallback to direct launching if authentication window fails
        print("Falling back to direct launch")
        app = QtWidgets.QApplication(sys.argv)
        MainWindow = MovableMainWindow()
        ui = Ui_MainWindow()
        ui.setupUi(MainWindow)
        MainWindow.show()
        sys.exit(app.exec_())
except Exception as main_error:
    # This catches any error in the main block
    print(f"Critical error in main: {main_error}")
    with open("critical_error.log", "w") as f:
        f.write(f"Critical error: {main_error}")

    # Last resort - extremely simple window launch
    try:
        import traceback
```

```
    traceback.print_exc()
    with open("traceback.log", "w") as f:
        traceback.print_exc(file=f)

    app = QtWidgets.QApplication(sys.argv)
    main_win = QtWidgets.QMainWindow()
    ui = Ui_MainWindow()
    ui.setupUi(main_win)
    main_win.show()
    sys.exit(app.exec_())
except Exception as last_error:
    print(f"Final fallback failed: {last_error}")
    with open("final_error.log", "w") as f:
        f.write(f"Final fallback failed: {last_error}")
```

Chapter 7: Conclusion and Future Scope

CONCLUSION

- The goal of Jarvis: The Lab AI Assistant is achieved, and it successfully automates task execution, voice recognition, and AI-powered assistance in laboratory environments. The system is designed to be user-friendly, interactive, and efficient.
- As AI technology continues to evolve, Jarvis demonstrates how intelligent assistants can enhance productivity and streamline operations in labs, classrooms, and professional workspaces.
- Implementing Jarvis in academic and research environments saves time and reduces the manual effort required for information retrieval, task management, and repetitive operations.
- The system provides hands-free, voice-controlled operations, ensuring users can focus on their work without distractions.
- With Google Gemini AI, Jarvis generates context-aware responses, improving accuracy and adaptability to different user needs.
- The GUI interface built with PyQt5 makes interactions more engaging and intuitive, allowing users to manage tasks and retrieve data easily.
- The project is customizable and scalable, meaning it can be adapted for various domains, such as corporate offices, research labs, educational institutions, and smart home automation.
- The log management system ensures task tracking and accountability, helping users keep a record of past interactions for improved workflow management.

FUTURE SCOPE

We can further enhance Jarvis: The Lab AI Assistant by adding the following features:

- Advanced NLP and AI Capabilities:
  - Implement deep learning models to improve natural language understanding and conversational AI.
  - Enable multi-turn conversations, where Jarvis remembers context from previous interactions.
- Integration with IoT and Smart Devices:
  - Enable control over lab equipment, lights, and other electronic devices using voice commands.
  - Connect with smart home and office automation systems (e.g., Google Home, Amazon Alexa).
- Cloud-Based AI and Data Synchronization:
  - Allow cloud-based voice processing for better accuracy and response speed.
  - Enable real-time synchronization of user preferences and command history across multiple devices.
- Multi-Language Support:
  - Expand Jarvis's capabilities to support multiple languages, improving accessibility for global users.
  - Add a real-time translation module to help in multilingual research environments.

References and Bibliography

Reference Links
Artificial Intelligence and Machine Learning

- Google Gemini API Documentation
  https://ai.google.dev/

- Speech Recognition using Python
  https://realpython.com/python-speech-recognition/

- Natural Language Processing (NLP) Guide
  https://towardsdatascience.com/nlp-guide

Programming and Libraries Used

- Python Official Documentation
  https://docs.python.org/3/

- PyQt5 GUI Development
  https://www.riverbankcomputing.com/software/pyqt/

- Pyttsx3 (Text-to-Speech)
  https://pypi.org/project/pyttsx3/

- SpeechRecognition Library
  https://pypi.org/project/SpeechRecognition/

Research Papers and Articles

- A Comparative Study on Virtual Assistants
  https://www.sciencedirect.com/science/article/pii/S1877050920304233

- Enhancing Human-Computer Interaction Through AI-Based Virtual Assistants
  https://ijcsit.com/AI-Assistants-HCI